QIK-VRT-Speichertechnologie: kanonische Daten, Rechtebindung und nachweisbare Wirkung

Informatik-Fachartikel | Autorenprüffassung vom 30. Oktober 2026

Konzept, Anforderungen, Product Ownership und Freigabe: Ingolf Lohmann. Technische Ausarbeitung, Implementierung, Formalisierung und Textentwurf: OpenAI Codex. Diese Beitragsangabe ist eine Herkunftserklärung; sie ersetzt keine urheberrechtliche Prüfung einzelner Beiträge. Dokumentlizenz: CC BY-NC-ND 4.0. Keine Zeitschriftenbegutachtung, DOI-Veröffentlichung oder produktive Abnahme wird behauptet.

Kurzfassung

QIK-VRT verbindet eine eindeutige Darstellung endlicher Daten mit Protokollidentität, Herkunftsangaben und einer kontrollierten Verarbeitungskette. Das hier ergänzte Rechteprofil persistiert in jedem darüber erzeugten Speicherelement die QIK-VRT-Zuordnung zu Ingolf Lohmann, den vollständigen PolyForm-Softwarelizenztext und eine getrennte Erklärung zu den Nutzdaten. Ein bestehender Decoder prüft die feste 400-Byte-Protokollsignatur, Längen, Ordnungsregeln, Inhaltsidentitäten und das Rechteprofil, bevor er Daten als gültig annimmt. Strukturierte Diagnosen erlauben die lokale Einbindung in weitere Systeme.

22 mit Lean 4.19 geprüfte Sätze beschreiben Integritäts- und Adaptationseigenschaften eines ausdrücklich abgegrenzten Modells. 69 lokale Codec- und Speicherelementtests prüfen die konkrete Python-Implementierung einschließlich 6.400 einzelner Signaturbitänderungen an einem Beispielcontainer. Die Formalisierung ersetzt keinen Beweis des Python-Parsers, des Hashverfahrens, der Betriebssysteme oder einer unbegrenzten automatischen Anpassungsfähigkeit. Das Effect Acknowledgement Protocol ergänzt den Speicher um eine getrennte Prüfung tatsächlicher Wirkungen.

1. Ein Speicherelement mit nachvollziehbarem Bezug

Die Nutzdaten eines QIK-VRT-Elements sind zunächst eine endliche Folge undurchsichtiger Bytes. Ihr Inhalt wird beim Speichern weder ausgeführt noch als Text normalisiert. Ein Bild, ein Programm, ein Messdatensatz und ein bereits komprimiertes Archiv können deshalb denselben Transportmechanismus benutzen. Die Bedeutung der Bytes entsteht durch einen zusätzlich festgelegten Interpretations- oder Ausführungsvertrag.

Der bestehende V1-Standpunkt enthält eine öffentliche, exakt festgelegte 400-Byte-Signatur. Ein Element verbindet diese Signatur mit einer vom Aufrufer dauerhaft vergebenen 32-Byte-Kennung und seinem Inhalt. Eine domänengetrennte SHA-256-Berechnung bildet die Inhaltsidentität. Eine Änderung am Inhalt erhält die logische Kennung, erzeugt aber eine neue Inhaltsidentität. Die Kennung allein ist keine globale Einmaligkeitsgarantie und keine Identität einer natürlichen Person.

Ein Snapshot ordnet seine Elemente nach ihren Kennungen. Doppelte Kennungen, unzulässige Längen, zusätzliche Bytes und abweichende Signaturen werden abgewiesen. Ein nachgeschalteter blockweiser Transport nutzt bei Bedarf zlib-Kompression. Das Verfahren garantiert keine Verkleinerung jeder Eingabe: Bei ungeeigneten Daten kann der Verwaltungsaufwand die Darstellung vergrößern. Verlustfreiheit und Kompressionsgewinn sind getrennte Eigenschaften.

2. Was doppelte Kanonizität hier bedeutet

Für einen geprüften Codec mit Encoder E und Decoder D sind zwei Beziehungen wesentlich:

1. D(E(x)) = x: Der gespeicherte Inhalt wird unverändert wiederhergestellt.
2. E(D(w)) = w für vom Encoder erzeugte kanonische Darstellungen w: Die Wiederherstellung und erneute Speicherung ergeben dieselben Bytes.

Die zweite Gleichheit wird hier für das Bild des jeweiligen Encoders beansprucht. Unterschiedliche Kompressionsprofile dürfen denselben Inhalt verschieden darstellen. Für den V1-Snapshot ohne diese Transportwahl legt dagegen der bestehende Binärvertrag eine feste Ordnung und Darstellung fest.

Die Lean-Struktur `CanonicalCodec` verlangt die erste Beziehung als explizite Voraussetzung. Daraus werden die zweite Beziehung auf erzeugten Darstellungen und die Injektivität des Encoders bewiesen. Das ist ein allgemeiner algebraischer Satz. Die Einsetzung des konkreten Python-Codecs in diese Struktur bleibt eine gesonderte Verifikationsaufgabe; die vorliegenden Implementierungstests liefern dazu empirische Softwareevidenz.

Die beiden Gleichheiten sind nützlich für Archivierung, Wiederaufnahme und Portierung. Sie machen Daten nicht selbst zu einem Prozessor. Eine Laufzeitumgebung benötigt zusätzlich einen Interpreter, eine Instruktionssemantik und einen zugelassenen Ausführungspfad. Ein selbstextrahierendes Archiv enthält seinerseits ein plattformabhängiges Startprogramm. Ein universeller Inhaltsvertrag ersetzt diese Startvoraussetzungen nicht.

3. Persistente Rechte und getrennte Nutzdatenherkunft

Das ergänzte Profil ist eine Erweiterung des vorhandenen Codecs. Jedes darüber erzeugte Element enthält einen eigenen Rechteumschlag innerhalb seiner gehashten Nutzlast. Der Umschlag bindet:

- Ingolf Lohmann als benannten QIK-VRT-Ursprungsgeber und Rechteinhaber der entsprechend lizenzierten QIK-VRT-Materialien;
- den vollständigen unveränderten Softwarelizenztext PolyForm Noncommercial 1.0.0 und den erforderlichen Hinweis;
- den Verweis auf die Dokumentlizenz CC BY-NC-ND 4.0;
- den Lizenzpreis von null Euro für erlaubte private nichtkommerzielle Nutzung;
- den Vorbehalt einer gesonderten schriftlichen kostenpflichtigen Vereinbarung für gewöhnliche kommerzielle Nutzung außerhalb der bestehenden Lizenzfreigaben;
- getrennte Angaben zu Urheberzuordnung, Lizenz und Quelle der jeweiligen Nutzdaten.

Der Preis einer kommerziellen Vereinbarung wird im Element nicht erfunden. Das Profil führt keine Zahlung aus und erzeugt keinen Vertragsschluss. Die vorhandenen PolyForm-Ausnahmen, konkrete Dateilizenzen, frühere Apache- und MIT-Freigaben sowie Drittrechte bleiben erhalten. Die Aufbewahrung fremder Daten in einem QIK-VRT-Element macht Ingolf Lohmann nicht zu deren Urheber. Die Herkunftsangaben der Nutzdaten sind zunächst Erklärungen des Erzeugers des Elements.

Die Rechtebytes werden exakt mit dem im Prüfer festgelegten Profil verglichen. Entfernt oder verändert jemand sie und berechnet anschließend den Inhalts-Hash neu, scheitert die Annahme weiterhin an diesem Profilvergleich. Ein alter V1-Snapshot wird nicht stillschweigend zu einem Rechteprofil hochgestuft. Historische Formate bleiben für ihren ursprünglichen Vertrag lesbar; eine

flächendeckende Migration sämtlicher vorhandener Speicher- und Hardwarepfade ist damit noch nicht erfolgt.

4. Ein Bit ändern: präzise Aussage statt absoluter Unverletzbarkeit

Die Signaturprüfung akzeptiert nur die festgelegten Signaturbytes einschließlich Auffüllung. Im Lean-Modell lässt sich jede Bitposition als beliebiger Anfang, das betroffene Bit und ein beliebiger Rest darstellen. Die Negation dieses einen Bits verändert die Folge. War die Signatur vorher gleich der vertrauenswürdig vorgegebenen Referenz, ist sie danach ungleich und wird abgewiesen. Der Satz ist vom Inhalt der Nutzdaten unabhängig. Ein entsprechender Satz gilt für die exakt gebundene Rechtefolge.

Für diese Aussage sind der Prüfer, seine Referenz und die vollständige Zuführung des zu prüfenden Elements vertrauenswürdige Voraussetzungen. Wird das ganze Programm gelöscht, seine Ausführung verhindert oder der Prüfer ersetzt, kann eine passive Datei kein eigenes äußeres Warnsignal erzwingen. Das Modell weist diese Grenze durch eigene Sätze aus. Ein ausgefallener Prüfer liefert den Status UNVERIFIED; eine beobachtete Abweichung liefert REJECTED.

Die öffentliche QIK-VRT-Signatur ist außerdem keine kryptografische Unterschrift mit geheimem Schlüssel. Wer beliebige Nutzdaten ersetzt und alle öffentlich berechenbaren Hashes neu erzeugt, kann weiterhin einen strukturell gültigen Container herstellen. Der Testfall zur vollständigen Nutzdatensubstitution zeigt genau das. Ein unabhängig vertrauenswürdig bezogener erwarteter Inhaltsdigest erkennt die Ersetzung unter der üblichen kryptografischen Sicherheitsannahme. Für stärkere Herkunftsauthentifizierung ist ein Signaturverfahren mit vertrauenswürdig zugeordnetem öffentlichem Schlüssel erforderlich. Selbst dessen Sicherheit und die Zuordnung zur Person sind eigene Nachweispflichten [2].

5. Diagnose ohne externen Dienst

Die Speicherprüfung funktioniert lokal. Weder ein Sprachmodell noch ein Netzwerkdienst muss der Extraktion zustimmen. Die Befehle `seal` und `unseal` nutzen den vorhandenen Transport; `unseal` veröffentlicht den wiederhergestellten Inhalt erst nach vollständiger Prüfung und atomarer Ersetzung der Ausgabedatei. Bei einem Fehler bleibt eine bereits vorhandene Zieldatei erhalten.

Die API `inspect_licensed` liefert einen strukturierten Diagnosebeleg. Er enthält den geprüften Inhaltsbezug, den Prüfstatus und den Zustand einer optionalen Meldeschnittstelle. Ein aufgerufener Callback kann die Diagnose an ein bereits zugelassenes Log-, Terminal- oder Überwachungssystem übergeben. Ein fehlerhafter Callback wird als SIGNAL_FAILED sichtbar. Ein erfolgreicher Callback-Aufruf ergibt lediglich SUBMITTED_UNCONFIRMED. Er beweist weder den Empfang an einem entfernten Ziel noch eine tatsächliche Wirkung dort.

Das Profil setzt deshalb `effect_ack_done` ausdrücklich auf false. Ein künftig ergänzter HTTP- oder Hardwareadapter muss Empfänger, Kanal, Berechtigung und Rücklesung selbst nachweisen. Die aktuelle Erweiterung installiert keinen neuen Netzwerkdienst und behauptet keine erfolgte externe Alarmzustellung.

6. Vom gültigen Inhalt zur nachgewiesenen Wirkung

Ein korrektes Archiv beantwortet die Frage, welche Bytes geprüft wurden. Eine Automatisierung muss zusätzlich beantworten, was mit diesen Bytes geschehen ist. Hier setzt das Effect Acknowledgement Protocol an.

Der vorhandene QIK-VRT-Entscheidungskern unterscheidet NACK, CONTINUE, DONE, ISOLATE und BLOCK. Eine normale Freigabe ist an DONE gebunden. Ein Transportbeleg allein genügt nicht. Aufgabe, Kontext, Berechtigung, erwartete Wirkung und beobachteter Rücklauf müssen zum selben Vorgang gehören.

Ein Softwareupdate verdeutlicht den Unterschied. Die Speicherprüfung kann feststellen, dass das Paket dem erwarteten Inhalt entspricht und seine Rechteangaben enthält. Ein Installationsprozess kann das Paket auf ein Ziel kopieren. Erst ein gebundener, hinreichend unabhängiger Readback kann belegen, dass dort die erwartete Version läuft. Eine erfolgreich gesendete Nachricht bleibt bis zu diesem Nachweis ein offener Vorgang.

Der Ansatz verbindet damit langlebigen Datenbezug mit überprüfbarer Handlung. Die Verlässlichkeit hängt weiter von den verwendeten Beobachtern ab. Ein Rücklesepfad, der denselben Fehler wie der Ausführer teilt, liefert keine vollständige Unabhängigkeit. Das Autoritätsrepository oder ein lokaler Test ersetzt keine Abnahme auf dem Produktionshost.

7. Lean: geprüfter Umfang und offene Übertragung

Die neue Datei `QIKVRTEffectAck/StorageIntegrity.lean` enthält 22 geprüfte Sätze. Ihr Audit weist keine Axiomenabhängigkeiten aus; insbesondere werden keine eigenen Axiome, `sorry` oder `native_decide` zur Begründung verwendet [1]. Drei absichtlich falsche Kontrollbehauptungen müssen vom selben Lean-Werkzeug abgewiesen werden.

Modellbereich	Nachgewiesene Eigenschaft	Voraussetzung oder Grenze
Signatur und Rechte	Einzelbitänderung und vollständige Entfernung einer nichtleeren Referenz scheitern	Exakter Vergleich mit unveränderter Referenz
Nutzdaten	Die Rechteprüfung ist unabhängig vom Nutzdateninhalt	Keine Beglaubigung des Inhalts durch den öffentlichen Standpunkt
Diagnose	Ausfall und erkannte Abweichung haben verschiedene Zustände	Tatsächlicher Prüferaufruf muss beobachtbar sein
Meldung	Ohne laufenden Prüfer oder verfügbaren Kanal kein garantierter Ausgang	Verfügbarkeit und Zustellung sind eigene Pflichten
Kanonischer Codec	Rekonstruktion, erneute Kodierung und Injektivität	Korrekte inverse Codecabbildung wird vorausgesetzt
Laufzeitanpassung	Jede endliche Folge eigenschaftserhaltender Schritte erhält die Invariante	Jeder Adapter muss seine Erhaltungsbedingung erfüllen

Diese Sätze prüfen eine mathematische Beschreibung. Lean bestätigt nicht automatisch, dass ein informeller Werbesatz dasselbe bedeutet wie die kodierte Aussage. Ebenso entsteht aus einem Kernelbeleg keine Verifikation des Python-Codes, des Compilers, eines FPGAs oder eines Quantenprozessors. Die Zuordnung zwischen Modell und Implementierung ist in der Claim-Matrix offen ausgewiesen.

8. Laufzeitpflege und der Begriff Superdeterminismus

Die Architektur kann DevOps-Werkzeuge verwenden, um neue Laufzeitversionen zu erkennen, einen Adapter oder Build vorzubereiten, Tests auszuführen und einen Kandidaten zur Annahme bereitzustellen. Eine automatisierte Fortsetzung innerhalb vorhandener Berechtigungen ist ein praktisches Entwicklungsziel. Die Lean-Induktion beschreibt dabei die Erhaltung einer Invariante über beliebig viele endliche, jeweils korrekte Schritte.

Sie beweist nicht, dass für jede unbekannte künftige Laufzeit ein geeigneter Adapter existiert, automatisch gefunden wird oder ohne menschliche Entscheidung eingesetzt werden darf. Maschinensprache allein garantiert weder genügend Speicher noch passende Ein-/Ausgabe, dokumentierte Semantik oder verfügbare Werkzeuge. Bereits eine Zielrepräsentation mit nur einem möglichen Zustand kann nicht einmal zwei verschiedene Boolesche Inhalte verlustfrei unterscheiden; auch diese Grenze ist formal bewiesen.

Ingolf Lohmann bezeichnet die angestrebte vollständig nachvollziehbare Steuerung als superdeterministisch. Für diesen Softwareentwurf wird das als Architekturbegriff für explizite Eingaben, festgelegte Regeln und reproduzierbare Entscheidungen eingeordnet. Ein physikalischer Superdeterminismus wird weder vorausgesetzt noch bewiesen. Zufällige oder unvollständig bekannte Messwerte können als Eingaben samt Herkunft und Unsicherheit gespeichert werden. Die Steuerentscheidung kann dadurch nachvollziehbar sein, ohne dass die Naturmessung deterministisch sein muss.

Auch die erfolgreiche Selbstanwendung ist ein konkreter Reproduktions- und Integrationstest. Dass ein System bei seiner eigenen Entwicklung hilft, bestätigt diesen beobachteten Ablauf. Daraus folgt kein zirkelfreier Beweis sämtlicher Aussagen über das System und seine künftigen Einsatzorte.

9. Hardware, Quantenrechner und analoge Daten

Ein gemeinsamer Daten- und Wirkungsvertrag kann unterschiedliche Rechenarten zusammenführen. Ein Quantenadapter müsste Schaltung oder Zwischenrepräsentation, Backend, Kalibrierung, Messproben, Unsicherheiten und Nachverarbeitung binden. Das im Repository vorhandene Modell `QuantumClassicalRuntime.lean` trennt bereits verantwortliche Freigabe von deterministischen Messergebnissen. Ein solches Modell ist kein angeschlossener Quantencomputer.

Analoge Quellen benötigen einen definierten Erfassungsweg. Bei Digitalisierung gehören Abtastzeit, Bandbreite, Quantisierung, Kalibrierung und Fehlergrenzen zur Bedeutung der gespeicherten Werte. Ein endliches digitales Element kann eine beliebige kontinuierliche Größe nicht automatisch mit unbegrenzter Genauigkeit abbilden. Es kann aber die verwendete Abbildung und ihre Grenzen nachvollziehbar festhalten.

Für eine Hardwarefassung des geprüften Vergleichs- und Zustandskerns sind beispielsweise Register, Vergleichslogik und begrenzte Zustandsautomaten denkbar. Die konkrete Realisierung müsste zusätzlich gegen ihr formales Modell verifiziert werden. Manipulationserkennung und äußere Meldung benötigen dort eine passende Vertrauenswurzel, Energieversorgung und einen überwachten Meldeweg. Eine FPGA-, ASIC-, Quanten- oder Analogintegration wurde durch diese Softwareerweiterung nicht gebaut oder abgenommen.

10. Technischer Stand und Reproduktion

Die Erweiterung baut auf Authority-Main `6fc9c45a272fb661f09bf5547e63e70e42d45271` mit Tree `7b705174f4a8d1948d884930ffa2e6cee6c23304` auf. Historische Signaturbytes und V1-Vektoren bleiben unverändert. Der konkrete Umfang ist in Work Unit, Claim-Matrix und lokalem Kernelbeleg gebunden.

Die 69 lokal ausgeführten Codec-Tests umfassen sowohl die bisherigen Formate als auch zwölf neue Rechteprofiltests. Sie prüfen unter anderem die 6.400 Signaturbitänderungen, Rechteentfernung nach Neuberechnung von Hashes, eine vollständige Nutzdatenersetzung, Längen- und Ressourcenfehler, ausgefallene Meldeschnittstellen sowie die atomare Kommandozeilenextraktion. Das ist überprüfbare Evidenz für diese Fälle; es ist kein vollständiger Beweis aller Programmeingaben oder aller Angreiferstrategien.

Die Nutzung erfolgt beispielsweise über `python3 -B -m src.qikvrt_codec seal` mit dauerhafter Kennung und expliziter Nutzdatenherkunft sowie anschließend `unseal`. Der technische README enthält die vollständigen Befehle. Das Lean-Modul lässt sich im vorhandenen Projekt mit `lake build QIKVRTEffectAck.StorageIntegrity` bauen. Anschließend prüft `lake env lean QIKVRTEffectAck/StorageIntegrityAxiomAudit.lean` die Abhängigkeiten. Die vorhandene CI wird um denselben Audit und genaue Quellenbindungen ergänzt.

Die Arbeit liefert damit einen lokal nutzbaren, teilbaren Softwarebaustein, einen abgegrenzten Modellbeweis und eine dokumentierte Anschlussmöglichkeit an das vorhandene Wirkungsprotokoll. Produktive Domain-Auslieferung, tatsächliche externe Meldungen und ein universeller selbsttätiger Hardwareanschluss bleiben gesondert zu belegen.

Quellen und zugehörige Nachweise

[1] Lean Reference Manual: Validating a Lean Proof. https://lean-lang.org/doc/reference/latest/ValidatingProofs/ (abgerufen 30.10.2026).

[2] NIST: FIPS 186-5, Digital Signature Standard, 2023. https://doi.org/10.6028/NIST.FIPS.186-5.

[3] PolyForm Project: PolyForm Noncommercial License 1.0.0. https://polyformproject.org/licenses/noncommercial/1.0.0. Maßgeblich für die im Profil benannten Softwarefreigaben bleibt der vollständige Lizenztext.

[4] Authority-Repository: https://github.com/ingolf-lohmann/qik-vrt. Implementierung: `src/qikvrt_standpoint_codex.py`, `src/qikvrt_codec.py`; Modell: `formalization/QIKVRT_Formalization_v2.0/QIKVRTEffectAck/StorageIntegrity.lean`; Prüfumfang: `CLAIM_MATRIX.json`, `LOCAL_KERNEL_RECEIPT.json` und README dieses Artikelpakets.